Public

HARDWARE MANAGEMENT



Advancing GPU & CPU Diagnostics and RAS Standardization for Hyperscale Infrastructure

Rama Bhimanadhuni, Principal Engineering Lead, Microsoft
Taniya Siddiqua, GPU RAS Architect, AMD

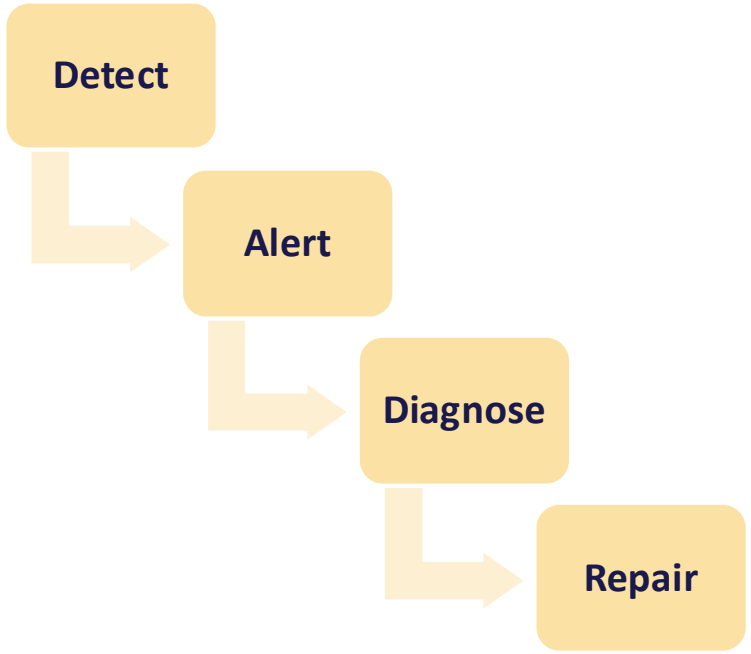
OCP PLATINUM Member

Public

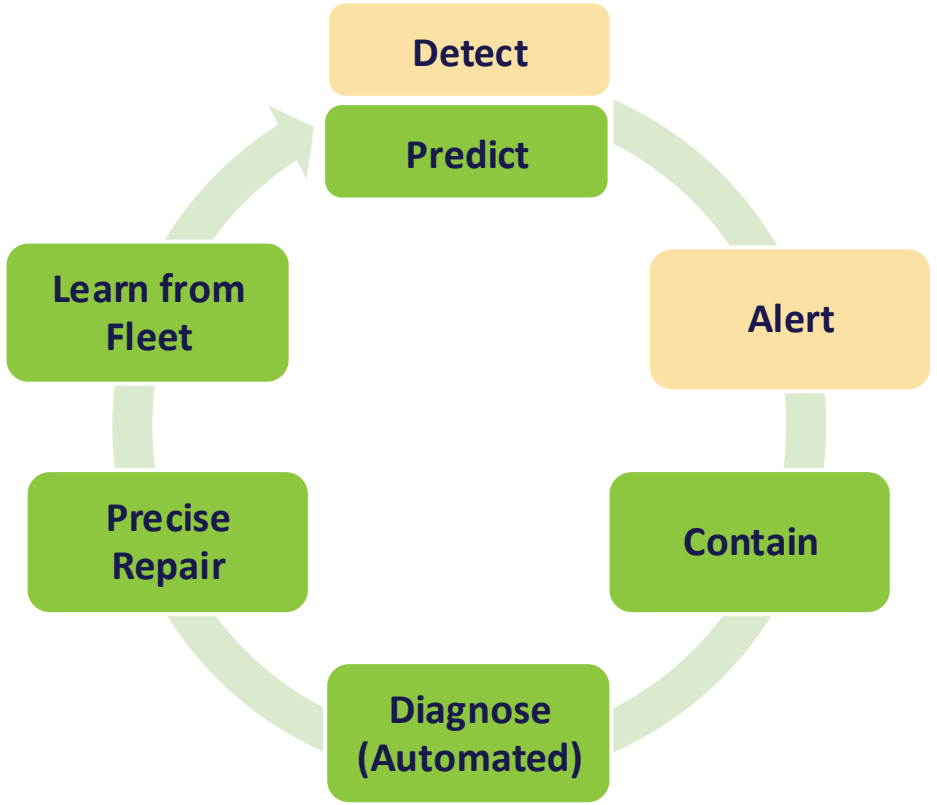
AI Fleets Need a New Operating Model

The KPI has shifted from compute deployed to useful intelligence per dollar and per watt, with a key focus on margin efficiency.

Yesterday's operating model does not work for the new KPI



New Operating Model



Public

Three Fleet Metrics That Matter

Turn reliability signals into measurable fleet outcomes.



01

CONTAIN

BLAST RADIUS

Contain a fault to one accelerator or node, not to an entire training job or rack.

Fault scope → job impact



02

DIAGNOSE

DIAGNOSTIC EFFICACY

Track detection rate, false positives and time-to-signal as fleet KPIs, not lab-only results.

Right signal • right time



03

REPAIR

MTTR (Mean Time to Repair)

Every minute a node is out of service costs useful work. Repair speed is an AI performance metric.

Days → Hour/Minutes

FLEET OUTCOME **More useful work completed with fewer disruptions and faster recovery.**

Contain → Diagnose → Repair

Public

OCP Standardization for Diag, Debug and RAS

Hyperscale GPU Management

- **OCP CLA Work Groups:** Formed by Microsoft, Google, NVIDIA, Meta and AMD.
- **Standardization Initiative:** First industry effort to standardize GPU RAS requirements
- **Progress:**
  - v1.7 published at 2025 OCP Global Summit
  - v1.9 published at 2026 OCP Global Summit

AMD Google Meta Microsoft NVIDIA

Hyperscale CPU Management

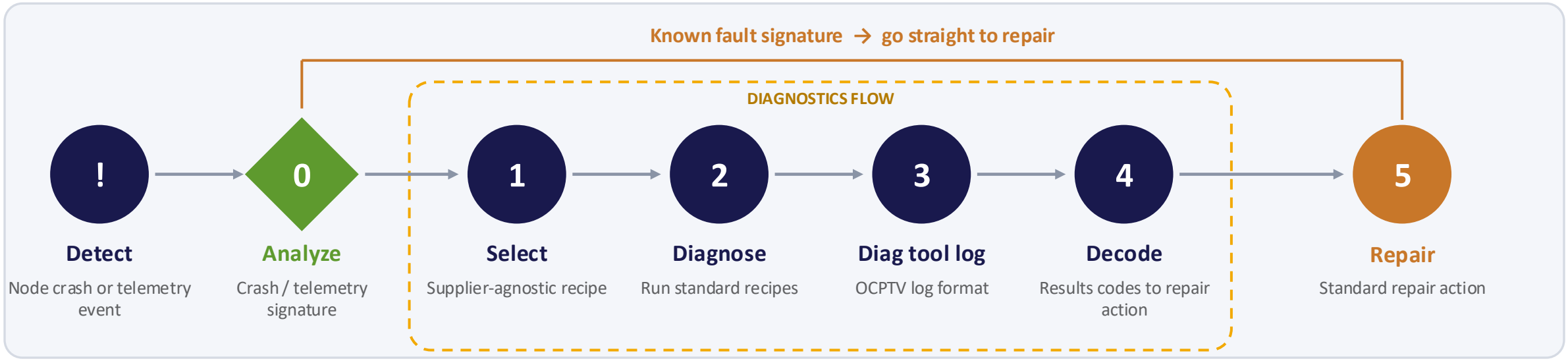
- **OCP CLA Work Groups: Formed by Microsoft, Google, AMD, Meta, ARM and Intel.**
- **Standardization Initiative: First industry effort to standardize CPU diagnostics and debug requirements**
- **Progress:**
  - v0.7 published at 2025 OCP Global Summit
  - v0.9 published at 2026 OCP Global Summit

AMD arm Google intel Meta Microsoft

Public

OCP Debug & Diagnostics

Reducing MTTR and fleet complexity through OCP-standardized debug and diagnostics



Supplier provided

- Fault analysis tools
- Error codes → repair actions, via JSON
- Diagnostic tools, recipes and OCPTV-standard logs

Hyperscaler managed

- Fleet telemetry & crash dumps
- Analysis + recipe selection, based on failure
- Repair action & FRU location, to tech or agent

✓ Heterogeneous fleet

✓ Faster root cause

✓ Better diagnosis

✓ Higher availability

Public

MTTR: Match Diagnostic Depth to the Fault

T0 – Telemetry-Based	T1 – Triage	T2 – Extended	T3 – Extensive
NO DIAGNOSTIC CYCLE	**5 TO 10 MINUTES**	**UNDER ONE HOUR**	**MULTI-HOUR**
Supplier documents telemetry signals and error patterns, and maps them to repair actions with confidence levels.	Fast go/no-go on every node entering repair, quick stress test that can drive repair steps.	Triggered on nodes that pass T1 but still show symptoms. Deeper component stress and functional validation.	Only for faults unresolved after T2. Full coverage equivalent to the complete test suite.

Conditional execution of recipes based on the fault to reduce MTTR across the fleet.

Public

Diagnostic efficacy -> Fleet KPI

90%+

Recommended: mature system, mature product

FR / (FR + FU)

resolved ÷ all observed failure events

AN EVENT COUNTS AS RESOLVED ONLY IF IT:

- ✓ Detects the failure
- ✓ Finds the repairable root cause
- ✓ Gives an actionable fix
- ✓ Restores normal operation

Metrics need to be measured per fault domain and in aggregate, with sample size and measurement period.

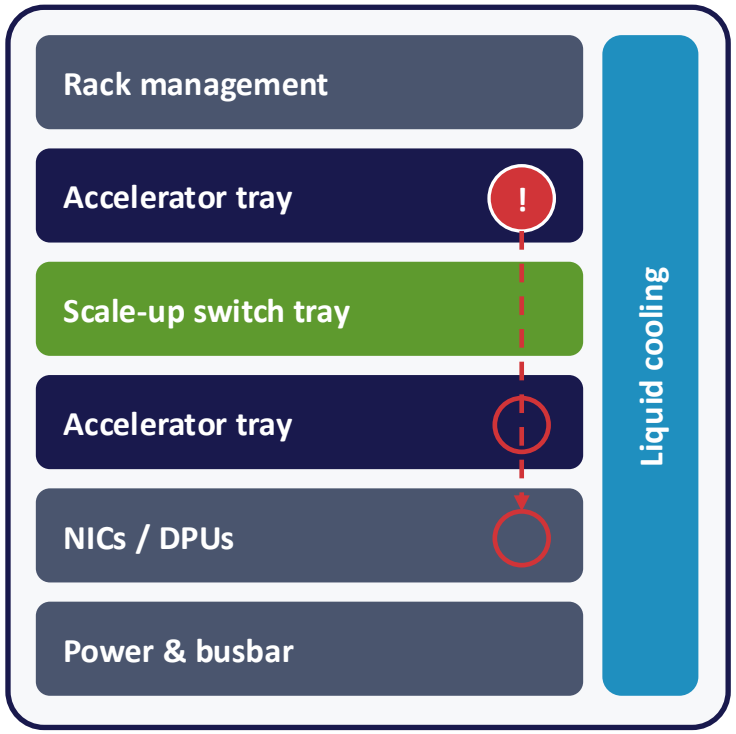
*FR - Resolved Failure Events, FU: Unresolved Failure Events

Public

From Node RAS to Rack-Scale RAS, Debug and Diagnostics

NODE Device-local errors → **RACK** Shared fault domains → **CLUSTER** Cascading impact

RACK FAULT DOMAINS



WHERE TODAY FALLS SHORT

01 Fragmented telemetry
Time-sync gaps across domains make root cause hard to correlate

02 Hard-to-isolate root cause
GPU, fabric, power and cooling faults cascade

03 Disconnected debug & diag
Vendor-specific flows slow fault isolation

PROPOSED OCP CONTRIBUTIONS

1 Common schema + time base

2 Cross-domain correlation

3 Rack-scale diagnostics + debug

Public

Blast Radius Reduction

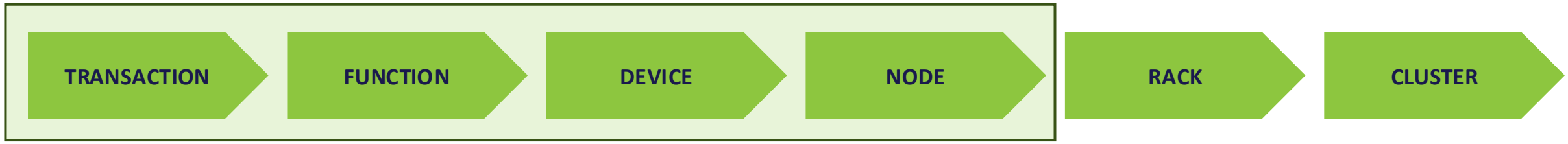
Contain a fault to one accelerator or node, not to an entire training job or rack.

WHY CHECKPOINT AND RESTART IS NOT ENOUGH

Large AI training clusters are tightly coupled, distributed and concurrent. Both checkpoint-save time and checkpoint-recovery time subtract from useful work, so restart alone cannot hold target cluster efficiency.

WHAT DEVICES SHOULD DO FIRST

Prioritize safe forward-progress mechanisms: retries, localized recovery, context recovery, rerouting, sparing, partitioning and degraded-mode execution, before requiring a full job restart.



RECOVER AT THE SMALLEST SAFE SCOPE

Public

Requirements for Blast Radius Reduction

CONTAINMENT FIRST

No mechanism may let corrupted data propagate silently; every recovery path stays consistent with error containment.

GRADUATED SCOPE

Multiple recovery scopes, from transaction retry to function reset to device reset, so software picks the least disruptive safe action.

ACTIONABLE TELEMETRY

When a mechanism is exercised, degraded or exhausted, log enough to drive automated mitigation and FRU isolation.

DOMAIN IDENTIFICATION

Devices and platforms provide enough information to identify the likely failed domain.

POLICY-DRIVEN

Hyperscalers choose recovery versus fail-fast by workload type and SLA objective.

Public

Recovery Techniques

Reduce fault blast radius by containing impact, preserving correctness, and restoring useful work at the smallest practical scope – values could be different for training vs inferencing.

1 **Legacy checkpointing & recovery**
Topology-agnostic software checkpoint/restart; broadly used, but software overhead can reduce cluster efficiency.

2 **Compute & context isolation**
Contain core or engine faults to a context or partition so unrelated work continues.

3 **Retries & replay**
Mask transient faults (e.g., memory-access or link-level retries) while preserving correctness.

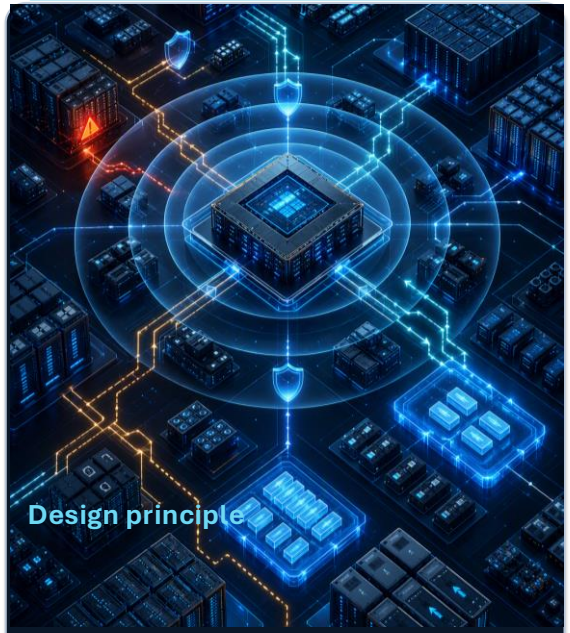
4 **Sparing & remapping**
Remap to cold or hot spares (e.g., rows, pages, lanes, cores, or routes) and keep device operational.

5 **Links & interconnects**
Recover or reroute GPU↔GPU, GPU↔CPU, and UBB-switch paths to avoid workload disruption.

6 **Software-assisted recovery**
Optimize checkpoint/restart and use poison-aware recovery paths to reduce recovery time and lost work.

7 **Degraded-mode operation**
Continue at reduced performance or capacity, with explicit notification to workload and infrastructure agents.

Outcome-based framework



Standardize outcomes and observability, while allowing suppliers to innovate in mechanism and implementation.

Public

Call to Action

- Provide feedback on 1.9 Version GPU RAS document - hyperscale-gpu@ocproject.net
- Provide feedback on 0.9 Version CPU Debug and RAS document
- Contribute to GPU RAS requirements and provides updates to Compliance tools
  - Compliance Tool Github Repo - opencomputeproject/ocp-diag-cpact
- Join the group discussion - Hardware Fault Management - Incubation » Open Compute Project

- Join the mailing list - https://ocp-all.groups.io/g/OCP-HWFaultMgt